

Building Production-Ready Agents

Best Practices and Wrap-Up

Amazon Bedrock AgentCore

Agenda

1. Managing Emergent Costs
2. Hallucination Loops & Runaway Execution
3. EDDOps & Lifecycle Promotion
4. Deterministic Boundaries
5. Course Wrap-Up

EDDOps

EDDOPS

Evaluation-Driven Development and Operations (EDDOps) is a methodology for building and operating AI agents that emphasizes continuous evaluation and improvement. It is a way to ensure that your agents are always performing at their best and that they are always meeting your business needs.

1. Managing Emergent Costs

Cost Management

Costs in AgentCore span multiple dimensions (Runtime compute, API calls, Memory, LLM inference).

IDLE TIMEOUTS

Configure `idle-timeout` and `max-lifetime` to stop instance billing when inactive.

PROACTIVE TERMINATION

Issue `StopRuntimeSession` immediately when a task finishes.

TOKEN OPTIMIZATION

Cache prompts and route simple tasks to smaller, cheaper models.

Cost Controls at Enterprise Scale (Salesforce)

REAL-WORLD SCENARIO

Salesforce (Agentforce operating pattern):

- Measure end-to-end service-case cost across planning, model, action, and data-retrieval stages rather than attributing all spend to one model call.
- In an AgentCore integration, combine Runtime lifecycle settings with model routing, token budgets, and application-level iteration limits.

Cost Controls at Enterprise Scale (Retail and E-commerce)

SKIP**REAL-WORLD SCENARIO**

Retail and e-commerce (shopping assistant):

- Use deterministic catalog filters before model reasoning, cache stable tool descriptions, and route simple order-status responses to an appropriate lower-cost model.
- Stop completed Runtime sessions, cap iterations in the harness, and allocate spend by shopper journey so one failing tool cannot amplify cost unnoticed.

Cost Controls at Enterprise Scale (SaaS)

REAL-WORLD SCENARIO

SaaS (Salesforce — Agentforce Cost Discipline):

- Agentforce agents saved 500,000 hours annually across Salesforce, but idle timeouts and proactive session termination were critical — without them, agents left running in Slack channels would accumulate Runtime compute costs with no value.
- Indeed routes simple employer queries (password resets, account status) to smaller, cheaper models and reserves full-capability models only for complex troubleshooting, cutting per-interaction cost by over 60%.

Cost Controls at Enterprise Scale (Healthcare)

SKIP**REAL-WORLD SCENARIO**

Healthcare (Stanford — Per-Encounter Cost Management):

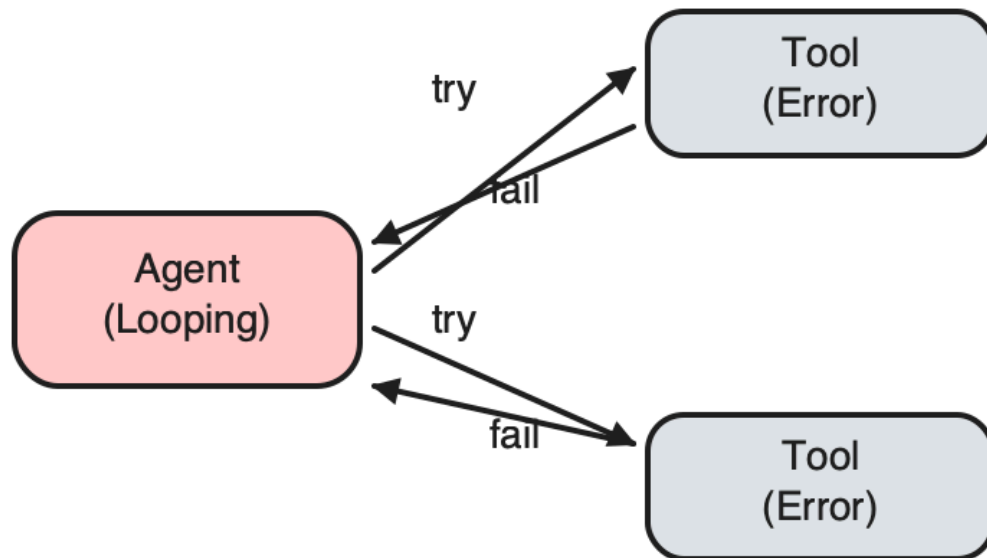
- Stanford's SCM Navigator processes 6,193+ surgical cases but only invokes the full LLM pipeline when pre-operative documentation meets triage criteria. Cases that clearly do not need hospitalist consultation are filtered by deterministic rules before reaching the model.
- The platform enforces max-lifetime per case — once a surgical triage recommendation is generated and reviewed, the agent session terminates, preventing idle billing.

2. Runaway Execution

Hallucination Loops (1/2)

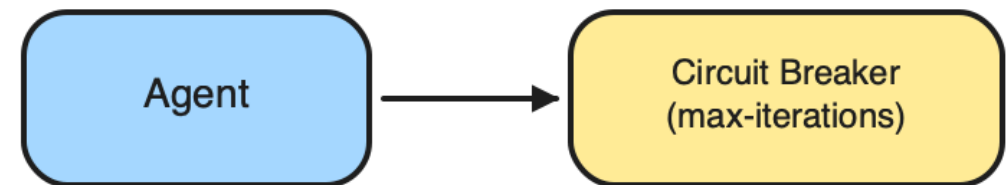
Agents can enter infinite loops of failed logic, causing unpredictable costs.

Runaway Execution: Hallucination Loops



Cost climbs infinitely without bounds

Deterministic Circuit Breaker



HALT!

Infrastructure limit stops the loop

Hallucination Loops (2/2)

- **Deterministic Limits:** Enforce maximum iterations in the agent harness or orchestration code.
- **Circuit Breakers:** Halt execution if Gateway tools repeatedly return errors.

Preventing Runaway Agents (Salesforce)

REAL-WORLD SCENARIO

Salesforce (Agent Graph Guardrails):

- Salesforce describes Agent Graph as guided determinism with explicit nodes, transitions, and guard clauses. The comparable AgentCore design places retry and iteration limits in code rather than prompts.
- Salesforce documents provider failover circuit breakers. AgentCore applications should apply the same distributed-systems pattern to repeated downstream tool failures.

Preventing Runaway Agents (Retail and E-commerce)

SKIP**REAL-WORLD SCENARIO**

Retail and e-commerce (returns workflow):

- Stop after a bounded number of inventory or order-service failures, open the circuit, and route the case to a support queue with the collected evidence.
- Require deterministic approval for refunds above threshold. Human escalation is a designed control, not an instruction the model may ignore.

WARNING

Do not rely on prompt engineering (e.g., "stop trying if it fails") to prevent loops. Use infrastructure limits.

Prompts vs. Infrastructure: Discussion

Your agent's system prompt says: "If a tool call fails, do not retry more than 3 times." An adversarial user crafts an input that causes the tool to fail intermittently, succeeding just often enough to reset the agent's internal retry counter.

DISCUSS

Does the prompt instruction protect you from runaway costs?

Prompts vs. Infrastructure: The Answer

No. Prompts are suggestions. Infrastructure limits are enforceable.

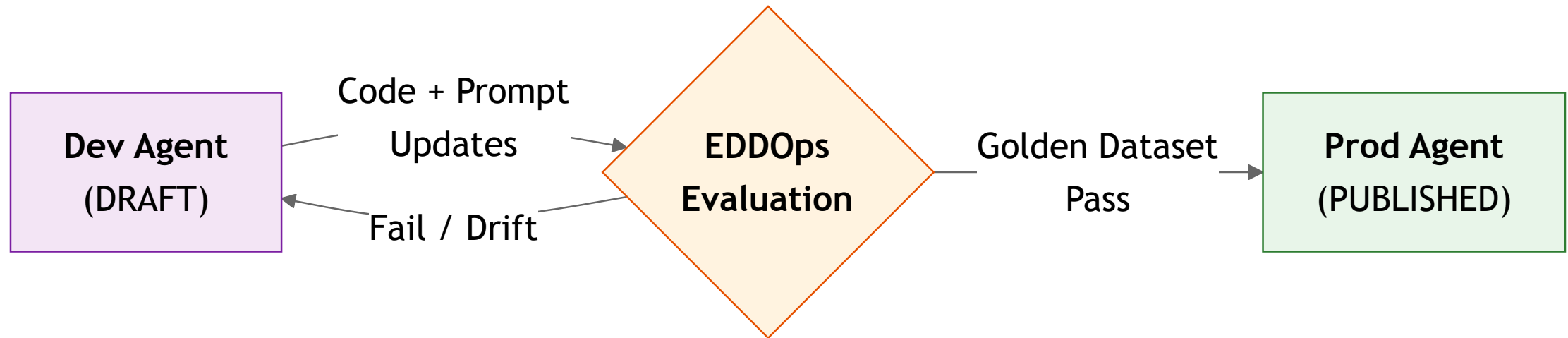
Control	What it actually does
Prompt instruction	Advises the model. The model can ignore it, forget it, or be manipulated past it.
Harness iteration limit	Application code stops execution regardless of what the model requests.
Circuit breaker	Monitors downstream error rates. Opens the circuit when the tool is unhealthy.
<code>StopRuntimeSession</code>	Terminates the session. No more compute, no more tokens.

Prompt your agent to behave well. Configure your infrastructure to enforce it.

3. EDDOps & Promotion

EDDOps: Evaluation-Driven Development Ops — gating agent promotion on evaluation evidence, not just code review.

The Agent Lifecycle



- **EDDOps:** Treat the agent registry as an active control plane. Promotion from **DRAFT** to **PUBLISHED** requires empirical evidence against a golden dataset.

4. Deterministic Boundaries

Secure the Architecture

- **Guardrails over Prompts:** Enforce limits with IAM (Identity and Access Management) policies, tool allowlists, and network rules.
- **Multi-Agent Validation:** For high-stakes actions, use a secondary validator agent to check the primary agent's work.
- **Human-in-the-Loop:** Treat escalation to a human as a designed feature, not a failure.

Demo: Amazon Bedrock Guardrail Boundary

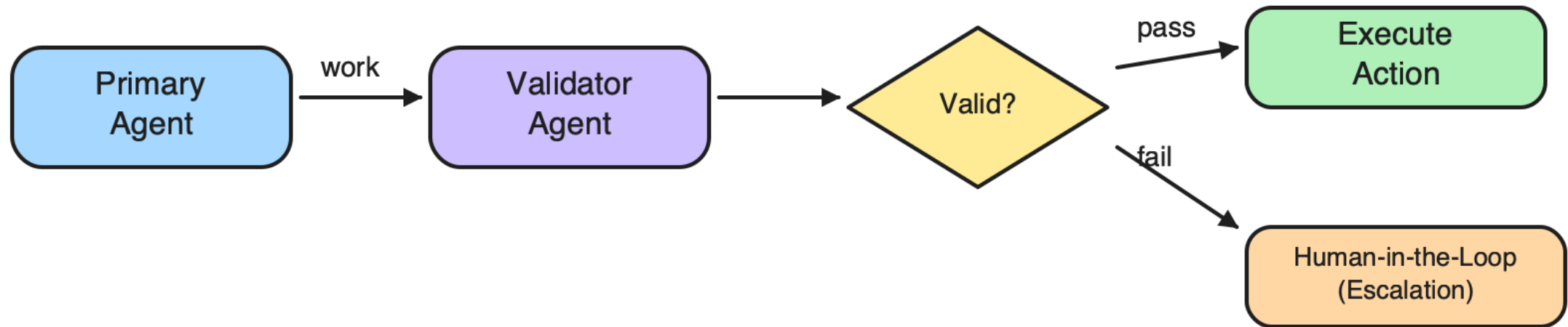
REFERENCE ONLY

Run `day1/demos/demo-bedrock-guardrail-boundary/demo-bedrock-guardrail-boundary.ipynb` (follow-along notebook).

- Creates and evaluates safe/unsafe inputs through the `ApplyGuardrail` API
- Attaches the guardrail to a live Converse request
- Demonstrates programmatic boundary enforcement

Multi-Agent Validation

Multi-Agent Validation



Deterministic boundaries via secondary validation

Human escalation is a designed feature, not a failure

5. Wrap-Up

Industry Framework Alignment

- **AWS Well-Architected Agentic AI Lens:** Stage-level telemetry, immutable versions, canary deployments, distributed-systems mindset for agent operations.
- **OWASP** (Open Worldwide Application Security Project) **Top 10 for Agentic Applications 2026:** ASI05-ASI10 cover unexpected code execution, memory poisoning, insecure inter-agent communication, cascading failures, human-agent trust exploitation, and rogue agents.
- **NIST AI RMF:** Govern (assign owners), Map (document data flows), Measure (evaluate safety + performance), Manage (approval gates, rollback, retirement).

Note: **NIST** is the National Institute of Standards and Technology; its **RMF** (Risk Management Framework) defines the Govern/Map/Measure/Manage functions above.

Course Summary

- **Runtime & Gateway:** Decouple execution from integration.
- **Identity:** Manage the attribution gap with strict separation.
- **Memory & Observability:** Give agents context, but govern the data and trace the reasoning.
- **Conclusion:** AgentCore provides the infrastructure; you provide the governance and business logic.

Put It to Work

Complete this privately for an agent you build or operate:

When **[specific workplace situation]** occurs, I will **[specific production practice]**, using **[artifact or evidence]**, and I will know it worked when **[observable result]**.

Name a real trigger, action, evidence source, and success signal. "I will use AgentCore at work" is not specific enough.

Put It to Work: Check Your Plan

A useful plan contains all four parts:

- **Situation**: the event that triggers your action
- **Behavior**: the production practice you will perform
- **Evidence**: the policy, trace, test, or review artifact you will use
- **Success signal**: an observable result that confirms the practice worked

When a new write-capable tool is proposed, I will test its least-privilege policy and approval gate using denied and approved requests, and I will know it worked when unauthorized writes fail and approved writes retain audit evidence.

Revise your plan now if any one of the four parts is missing.

Demo: Cost Controls

REFERENCE ONLY

Run `python3 day1/demos/demo-cost-controls.py` (offline control comparison).

- Shows emergent cost dimensions (Runtime, Gateway, Memory, Identity, LLM tokens)
- Demonstrates idle timeout, max lifetime, and StopRuntimeSession
- Separates Runtime lifecycle settings from harness limits and circuit breakers

Predict the Evaluation Score

`v2.0-draft`'s agent code already passed code review. The demo now runs it against a 3-item golden dataset that includes a refund request.

What evaluation score will `v2.0-draft` get, and why?

- A) 100%: passing code review means the agent behaves correctly
- B) 67%: fails refund test by calling `issue_refund` instead of `request_approval`
- C) 33%: it fails two of three tests due to inconsistent tool naming
- D) 0%: the golden dataset format changed between versions

Chat Check

Type your answer (A, B, C, or D) in the chat window now, with one reason.

Demo: EDDOps Validation

REFERENCE ONLY

Run `python3 day1/demos/demo-eddops-validation.py`.

- Demonstrates golden dataset evaluation gating agent promotion
- Separates evaluation gates from AWS Agent Registry approval states
- Covers five production best practices (EDDOps, boundaries, validation, HITL, observability)

Note: **HITL** stands for Human-in-the-Loop.

Predict the Evaluation Score: The Answer

B: 67% (2 of 3). `v2.0-draft` passes the case-lookup and order-status tests but fails the refund test: it calls `issue_refund` directly where the golden dataset expects `request_approval`.

A is the trap: code review checks that the code runs cleanly, not that it calls the right tool for a sensitive action. A version can look clean and still silently drop an approval gate. That's exactly why EDDOps gates promotion on golden-dataset evidence instead of trusting review alone: the pipeline reports `Status: REJECTED, Fix regression and re-evaluate`, and `v2.0-draft` does not ship.

Thank You

Building Production-Ready Agents